



The Karatsuba algorithm

The Karatsuba algorithm is a fast multiplication algorithm that was discovered by Anatolii Alexeevitch Karatsuba in 1960. It is used to multiply two n-digit numbers efficiently, reducing the number of multiplications required compared to traditional long multiplication.

The algorithm works by dividing the given numbers into smaller parts and recursively multiplying these parts to obtain the final result. The key insight of the Karatsuba algorithm is that the product of two n-digit numbers can be expressed as:

$$xy = (a * 10^{(n/2)} + b) * (c * 10^{(n/2)} + d)$$

where a, b, c, and d are smaller parts of x and y.

By applying this equation and using some mathematical transformations, the algorithm performs three multiplications instead of four, reducing the overall complexity.

Here is a high-level description of the Karatsuba algorithm:

1. Given two numbers, x and y , split them into two parts of roughly equal size: $x = (a * 10^{(n/2)}) + b$ $y = (c * 10^{(n/2)}) + d$
2. Recursively compute the following three multiplications: i. Compute $ac = a * c$ ii. Compute $bd = b * d$ iii. Compute $(a + b) * (c + d) = ac + ad + bc + bd$ (using only three multiplications)
3. Compute the final result: $xy = (ac * 10^n) + ((ad + bc) * 10^{(n/2)}) + bd$

The algorithm reduces the number of recursive multiplications by exploiting the properties of elementary algebra. It's particularly efficient when multiplying large numbers with many digits.

However, it's worth noting that for small inputs or on modern hardware with efficient multiplication instructions, the traditional long multiplication algorithm might perform better due to its simplicity and lower overhead.

The Karatsuba algorithm has been further optimized and extended in various ways, and it serves as a foundation for more advanced multiplication algorithms used in modern computer systems.

```
1. public class KaratsubaMultiplication {
2.     public static long karatsubaMultiply(long x, long y)
3.     {
4.         if (x < 10 || y < 10)
5.         {
6.             return x * y;
7.         }
8.         int n = Math.max(Long.toString(x).length(), Long.toString(y).length());
9.         int half = (n + 1) / 2;
10.
11.         long a = x / (long) Math.pow(10, half);
12.         long b = x % (long) Math.pow(10, half);
13.         long c = y / (long) Math.pow(10, half);
14.         long d = y % (long) Math.pow(10, half);
15.
16.         long ac = karatsubaMultiply(a, c);
17.         long bd = karatsubaMultiply(b, d);
18.         long adbc = karatsubaMultiply(a + b, c + d) - ac - bd;
19.
20.         return (long) (ac * Math.pow(10, 2 * half) + adbc * Math.pow(10, half) + bd);
21.     }
22. }
```

```
1. public static void main(String[] args) {  
2.     long x = 12;  
3.     long y = 2;  
4.     long product = karatsubaMultiply(x, y);  
5.     System.out.println("Product: " + product);  
6. }  
7. }
```

In the context of the Karatsuba algorithm, z_0 , z_1 , and z_2 are intermediate results obtained during the calculation of the product of two numbers. These values are derived from the formula:

$$xy = (z_2 * 10^{(2 * m)}) + ((z_1 - z_2 - z_0) * 10^m) + z_0$$

where x and y are the input numbers, m is the number of digits in the smaller number (usually half the total number of digits), and z_0 , z_1 , and z_2 are defined as follows:

$$z_0 = ac$$

$$z_1 = (a + b)(c + d) - ac - bd$$

$$z_2 = bd$$

Let's break down these intermediate results:

1. z_0 : It represents the product of the lower-order parts of x and y (bd).

$$z_0 = bd$$

2. z_1 : It represents the cross products of the sum of the parts $(a + b)$ and $(c + d)$ minus the individually calculated values of z_0 and z_2 .

$$z_1 = (a + b)(c + d) - ac - bd$$

3. z_2 : It represents the product of the higher-order parts of x and y (ac).

$$z_2 = ac$$

Using these intermediate results, the final product xy is calculated as a combination of the three terms, each shifted by the appropriate number of digits.

$$xy = (z_2 * 10^{(2 * m)}) + ((z_1 - z_2 - z_0) * 10^m) + z_0$$

The Karatsuba algorithm recursively calculates z_0 , z_1 , and z_2 until reaching a base case where simple multiplication can be performed for small numbers.

By exploiting the properties of algebra and breaking down the multiplication process into smaller parts, the Karatsuba algorithm reduces the number of multiplications required, leading to more efficient multiplication for larger numbers.



THANK YOU